

Week 1 • First React app: components, props & state

Technical Interaction Design • ITU • Autumn 2026

Built 11 September 2026 • commit bc3510e

Latest version: <https://itu-tid.github.io/lecture-notes-pdf/Week-01.pdf>

CONTENTS

Introduction to React

- React is a *component-based* UI library
- JSX is a combination of JS and HTML
- Interpolating JS in JSX
- Parameterizing Components
- Rendering Lists
- Event Handling

Component State

- Every component can store local state
- State comes from `useState`
- Reactive Programming
- What came up in the lecture

React Hooks

- A hook is how a component reaches things React is holding for it
- They all start with `use`
- The rules: top level, always, unconditionally

Introduction to React

Source: [Lectures/Technical/React/Intro-to-React.md](#) — something unclear? [Open an issue](#), or send a pull request.

React is a JavaScript **library** for building interactive **single page applications**. We will talk about what Single Page Applications are and how they work in a different lecture. For now, we want to be users of react so we'll start doing a very simple application in it and learn React on the way.

We build a to-do list as we go. Each section below is the smallest step that makes the previous step's problem go away, so the page reads in the order the app needs — and each one opens by saying what the app does at that point.

React is a *component-based* UI library

Everything in React is a component

A button is a component. A list row is a component. So is the whole page. You build a screen by nesting small ones inside bigger ones, and there is no other unit — no templates, no partials, no widgets.

Components are JS functions that return JSX elements

[Example](#) of a React component :

```
export default function TodoList() {  
  return (  
    <>  
      <h1>My To-Do</h1>  
      <ul>  
        <li>Buy milk</li>  
      </ul>  
    </>  
  );  
}
```

A plain JavaScript function, returning something that looks like HTML. The name is capitalised — React uses that to tell your components apart from built-in tags like `<ul>`, so `todoList` would not work.

Something unclear in this section? [Open an issue](#).

JSX is a combination of JS and HTML

The app now. The same heading, wrapped in a little markup — and the first surprise, because that markup is not quite HTML.

JSX is acronym for JavaScript + XML

A syntax extension for JavaScript

It lets you describe a UI declaratively, inside the language itself, rather than in a separate template file.

- syntax extension for Javascript!
- has the full power of JS inside of the {}
- template-like because it looks like other front-end JS frameworks (e.g., [Vue templates](#)) and server-side rendering frameworks (e.g. Jinja for Flask, Moustache)
- HTML-like code within JavaScript
- JSX is **transpiled** to Javascript

JSX has a stricter syntax than HTML

A component can only ever return a single JSX tag

Otherwise, you get an error.

```
// This won't work - multiple top-level elements
return (
  <h1>Title</h1>
  <p>Paragraph</p>
);

// This works but adds unnecessary <div>
return (
  <div>
    <h1>Title</h1>
    <p>Paragraph</p>
  </div>
);

// This works without extra DOM node
return (
  <>
    <h1>Title</h1>
    <p>Paragraph</p>
  </>
);
```

`<>...</>` is called a **fragment**: a wrapper that groups elements without adding anything to the page. Use it whenever you need a single parent but do not want a real `<div>` in the output.

Tags must always be closed

JSX converts most HTML and CSS attributes to camelCase

HTML event handlers, e.g. `onclick` becomes `onClick`

CSS attributes, e.g. `background-color` becomes `backgroundColor`

Special JS keyword `class` becomes `className`

Two minor exceptions from this rule.

Something unclear in this section? Open an issue.

Interpolating JS in JSX

The app now. The heading counts: *My To-Do (3)*. The number has to come from JavaScript, and it has to land inside the markup.

Curly brackets to escape JS inside JSX can be used in three ways

Inside text

```
return (  
  <  
    <h1>Hello {user.firstName + " " + user.lastName}</h1>  
    <p>Today is: {todaysDate}</p>  
  </>  
>);
```

As an attribute value

```
return (  
  <img  
    className="avatar"  
    src={user.imageUrl}  
  />  
>);
```

Double curlies, for objects

An object *inside* the braces — so `{{ }}`, which is one pair for the JS and one for the object. See [double curlies](#).

```
export default function TodoList() {  
  return (  
    <ul style={{  
      backgroundColor: 'black',  
      color: 'pink'  
    }}>  
      <li>Improve the videophone</li>  
      <li>Prepare aeronautics lectures</li>  
      <li>Work on the alcohol-fuelled engine</li>  
    </ul>  
  );  
}
```

Something unclear in this section? Open an issue.

Parameterizing Components

The app now. One `<TodoItem text="Buy milk" />`. Writing a second one by hand would mean copying the whole component, so it has to take its text from outside.

A component is a function

Functions can be parameterized, so components should be parameterizable too.

Parameters are called **props**

- the term very likely comes from properties

Passed like HTML attributes

- In the moment when the *props* are passed to the component, they *look* like HTML attributes – have the same syntax

They arrive as one object

- In the component definition the props are accessed as either
 - a single function parameter named `props`
 - a destructured dictionary

Or destructure them

Naming the props you expect in the signature itself — destructuring — makes the component easier to read and to write.

Here is our to-do row, written both ways:

```
// the props object, whole
function TodoItem(props) {
  return <li className="todo-item">{props.text}</li>;
}

// destructured – the version we will use from here on
function TodoItem({ text }) {
  return <li className="todo-item">{text}</li>;
}
```

Both are the same component. The second says, in its own signature, exactly which props it expects — so you can tell what it needs without reading the body. Used like this:

```
<TodoItem text="Buy milk" />
<TodoItem text="Call the landlord" />
```

Which is already tedious for two items, and impossible for a list that changes while the app is running. That is the next section.

With the `{children}` prop

Props pass **data** into a component. `children` passes **markup** into it — everything written between the opening and closing tags.

Start without it. A panel that takes its content as an ordinary prop:

```
function Panel({ title, text }) {
  return (
    <section className="panel">
      <h2>{title}</h2>
      <p>{text}</p>
    </section>
  );
}

<Panel title="Today" text="Nothing due." />
```

That works exactly as long as the content is one string. The moment you want a list inside the panel, or a button, or another component, you are stuck: you cannot put a `<ul>` inside a string. You would end up adding `items`, then `buttonLabel`, then `showIcon`, and the panel would slowly learn about everything that might ever go in it.

`children` is the way out. The component stops describing its content and only describes its *frame*:

```
function Panel({ title, children }) {
  return (
    <section className="panel">
      <h2>{title}</h2>
      {children}
    </section>
  );
}
```

Now the same component takes anything:

```
<Panel title="Today">
  <ul>
    <li>Buy milk</li>
    <li>Call the landlord</li>
  </ul>
</Panel>

<Panel title="Nothing due">
  <p>Enjoy the afternoon.</p>
  <button>Add something anyway</button>
</Panel>
```

Note what `Panel` does **not** know: that there is a list, that there is a button, that either exists. It knows it has a title and a frame to draw around whatever it was handed. That is the whole idea, and it is why almost every layout component you write — containers, cards, modals, page wrappers — ends up taking `children`.

The rule of thumb. If it is data the component needs to *use* — a title, a count, a user — make it a prop. If it is markup the component only needs to *place*, use `children`.

Something unclear in this section? Open an issue.

Rendering Lists

The app now. Three items, drawn from an array rather than typed out: the array is the app, and the screen is a picture of it.

Most applications sooner or later rely on lists of things that you want to process.

Lists are rendered with `array.map()`

Our three hardcoded `<TodoItem />` s become one array and one `map` :

```
const todos = ["Buy milk", "Call the landlord", "Book the dentist"];

function TodoList() {
  return (
    <ul>
      {todos.map((text, index) => (
        <TodoItem key={index} text={text} />
      ))}
    </ul>
  );
}
```

`map` turns an array of **strings** into an array of **JSX elements**, and React renders an array of elements by rendering each one. That is the whole mechanism. Add a fourth string to `todos` and a fourth row appears; nothing else has to change.

Every item needs a `key`

- must be unique **among its siblings**
- can be the database ID, UUID, or anything else stable
- it is how React tells the items apart between renders — without it, it cannot know whether you added a row, removed one, or reordered them
- if you don't do this, your console fills up with warnings

We are using the array **index** above, which is what everyone reaches for. While each row is nothing but text, it is genuinely fine: delete a row and React re-renders the list correctly, and you will see nothing wrong.

It stops being fine the moment a row holds something of its own that React is expected to keep — text typed into an input on that row, which input has the cursor in it, a checkbox the row tracks itself. Then the key is how React decides *which row is which* between renders, and an index is a lie about that. Delete the first of three and the row that was index 1 becomes index 0, so React reads it as “the thing with key 0 changed its text” rather than “the first thing is gone” — and whatever the old row 0 was holding stays behind on the row that took its place.

Put an input on every row and delete from the middle, and you can watch it happen. Until then the reason to give each to-do a real id of its own is a simpler one, and it arrives next week.

Nice examples of rendering lists and filtering at: *Describing the UI* > [Rendering Lists](#). Also nice exercises at the bottom of the page.

Something unclear in this section? [Open an issue](#).

Event Handling

The app now. An **Add** button that does nothing at all. Making it do something is the next two sections.

Interactive apps handle events

Click, type, mouse move, touch. Responding to these is the main job of a UI.

Event handlers are defined inside components

Handlers usually have names starting with `handle`

Built-in elements come with built-in events

Associating event handlers with events is done with the attribute in curly brackets syntax, as above. (See [onClick example](#).)

Our button, doing nothing useful yet:

```
function TodoList() {
  function handleAdd() {
    console.log("Add was clicked");
  }

  return (
    <
      <ul>
        {todos.map((text, index) => <TodoItem key={index} text={text} />)}
      </ul>
      <button onClick={handleAdd}>Add</button>
    </>
  );
}
```

Passing a function, not calling it

```
<button onClick={handleAdd}>Add</button> // hands React the function
<button onClick={handleAdd()}>Add</button> // calls it now, hands React the result
```

The second one runs `handleAdd` while the component is *rendering*, before anybody has clicked anything, and gives `onClick` whatever it returned — usually `undefined`. If your handler fires once on load and never again, this is why.

Handlers receive an event object

The `event` argument details info about what just happened.

- Sometimes you can ignore it,
- Sometimes you inspect it to learn about the event (e.g. mouse position, element that was clicked, etc.)

Events bubble up the DOM tree

If you have an `onClick` handler on both a button and a containing div, both will be handled in sequence, from the inner one outwards.

You meet this for real the moment each row gets a delete button:

```
<li onClick={() => toggleDone(id)}>
  {text}
  <button onClick={() => remove(id)}>×</button>
</li>
```

Click the × and *both* handlers fire — the button's, then the row's — so the item is deleted and the row you just deleted is also toggled. `e.stopPropagation()` inside the delete handler is the fix, and it is much easier to remember once you have watched it happen. [See event propagation example](#).

- Sometimes you can change the behavior of the event by calling `stopPropagation` or `preventDefault` on the event object. [example of stop propagation](#) and of [preventing default behavior](#).

Something unclear in this section? [Open an issue](#).

Exam Questions

1. What is JSX and how does it differ from HTML?

2. Why do we need to use a `key` attribute when rendering lists in React?

3. What is wrong with this component?

```
function Greeting() {
  return (
    <h1>Hello</h1>
    <p>Welcome to React</p>
  );
}
```

References

Read up from the react.dev documentation site, the following:

- Describing the UI
 - [Your First Component](#)
 - [Writing Markup with JSX](#)
 - [JS in JSX](#)
 - [Passing Props](#)
 - [Rendering Lists](#)
- Adding Interactivity
 - [Responding to Events](#)

Component State

Source: [Lectures/Technical/React/Component-State.md](#) — something unclear? [Open an issue](#), or send a pull request.

The app now. Add pushes a task onto the array — and nothing appears on screen. The variable really did change; the screen simply does not care. That failure is the whole argument for state, so it is worth running before you read on.

Every component can store local state

The difference from props is **ownership**. Props arrive from the parent and the component may only read them; state belongs to the component itself, which is the only thing that can change it. Props are what you were given; state is what you keep.

First, the version that does **not** work — worth typing out, because the reason it fails is the reason state exists:

```
const SAMPLE_TASKS = [
  "Buy milk",
  "Call the landlord",
  "Book the dentist",
  "Water the plants",
  "Reply to Mette"];

function randomTask() {
  return SAMPLE_TASKS[Math.floor(Math.random() * SAMPLE_TASKS.length)];
}

let todos = ["Buy milk"]; // an ordinary variable

function TodoList() {
  function handleAdd() {
    todos.push(randomTask());
    console.log(todos); // the array really does grow - look at the console
  }
  // ...and the screen never changes
}
```

The array grows. The console proves it. The screen ignores you completely, because nothing told React that anything happened — it only re-runs your component when you ask it to, and pushing onto a variable is not asking.

Something unclear in this section? [Open an issue](#).

State comes from `useState`

The `useState` hook:

- takes an **initial** value
- returns **current value** and a **setter function** (returns from where? you get it from React! React gave you this setter! It baked an observer inside of it! So now if you change the state with its help, it will know that the state has changed! Let's see what does this imply)

```
// We are importing the useState function from React
import { useState } from "react";

function TodoList() {
  const [todos, setTodos] = useState(["Buy milk"]);

  function handleAdd() {
    setTodos([...todos, randomTask()]);
    // a *new* array, not a push
  }

  return (
    <ul>
      {todos.map((text, index) => <TodoItem key={index} text={text} />)}
    </ul>
    <button onClick={handleAdd}>Add</button>
  );
}
```

Two things changed, and both matter:

1. `todos` now comes from `useState`, so React is the one holding it. The broken version above did survive — a variable outside the component persists perfectly well — but nothing connected it to the screen. Had we moved that `let` *inside* the component it would have been worse still, created fresh on every call.
2. `setTodos` is given a **new array** — `[...todos, randomTask()]` — rather than the old one mutated. **React decides whether to redraw by comparing what it was given against what it had** — not the contents, but whether it is *the same array*, the same one thing in memory. Hand back the array you already gave it and it sees no change, however different the contents. `push` would have done exactly that.

Optional: See the [button with counter example](#) for a combination of state and events

Something unclear in this section? Open an issue.

Reactive Programming

The app now. It works: **Add** appends a random sample task and the list grows. The question left over is *why the screen redrew at all* — nobody told it to.

When state changes, the component updates automatically; similar to Excel

When a state variable defined with `useState` changes with the help of the setter (and thus, not changing the variable directly!!) a redrawing of the whole component is triggered.

This is *reactive programming*. And reactive programming is why React is called so.

A bit like in Excel – one of the classical reactive programming environments – where when you change one cell, all the others who depend on it and only those are changed automatically.

In React, the dependents are not formulas, but UIs. When a state variable or a prop changes, the library automatically redraws all the relevant UI elements, and only those.

So the loop the app now runs is:

you click **Add** → `handleAdd` calls `setTodos` with a new array → React notices the state it is holding is not the array it had → it calls `TodoList()` again → `map` builds one more `<TodoItem />` than last time → the new row appears.

Nobody wrote “add a row to the screen” anywhere. You changed the data and described what the screen should look like for any data; React did the rest. That is the trade React asks you to make, and everything else this term is a consequence of it.

Here is the whole thing:

```
import { useState } from "react";

const SAMPLE_TASKS = ["Buy milk", "Call the landlord", "Book the dentist",
  "Water the plants", "Reply to Mette"];

function randomTask() {
  return SAMPLE_TASKS[Math.floor(Math.random() * SAMPLE_TASKS.length)];
}

function TodoItem({ text }) {
  return <li className="todo-item">{text}</li>;
}

export default function TodoList() {
  const [todos, setTodos] = useState(["Buy milk"]);

  function handleAdd() {
    setTodos([ ...todos, randomTask() ]);
  }

  return (
    <div>
      <h1>My To-Do ({todos.length})</h1>
      <ul>
        {todos.map((text, index) => (
          <TodoItem key={index} text={text} />
        ))}
      </ul>
      <button onClick={handleAdd}>Add</button>
    </div>
  );
}
```

Forty lines, and every idea above is in there somewhere.

And deliberately silly: **Add** picks a task at random, because there is nothing to type into yet. Giving it something to type into is [Forms and Controlled Components](#) — along with the first real bug, when deleting from the middle of the list shows you why keying by position was never going to hold.

Something unclear in this section? Open an issue.

What came up in the lecture

Things that happened while this was coded live, rather than things that were planned. They are here because they were the parts people remembered.

Asked in class: can a component change a prop?

Predict before you read on. Your component is handed `todos` as a prop. Can it change that array?

The lecture voted, and the large majority said no, on the grounds that props are read-only. Then we tried it:

```
function TodoList({ todos }) {  
  function handleAdd() {  
    todos.push("Buy milk");  
    console.log(todos);    // the array really did grow  
  }  
  // ...and the screen does not change  
}
```

No error, no warning, and the array is genuinely one longer. React freezes the props *object* while you are developing, so `todos = []` would fail — but the array **inside** it is not frozen, and `push` reaches straight into it.

So the honest statement is not *you cannot*. It is **you must not**, and two different things go wrong when you do.

The screen does not update, for the same reason the plain variable at the top of this note did not: nothing told React that anything had happened.

And worse, and invisibly: that array belongs to the **parent**. You have reached into another component's data and changed it without telling it. The parent still believes it holds the old list, and will go on rendering from that belief until something else happens to make it re-render — at which point your change appears, from nowhere, for no reason anyone can trace.

The reason this is worth two minutes is that it is how React works throughout. **The rules are promises you keep, not walls React builds.** You meet the same shape again with state: nothing physically stops you editing an object you already put there, and you must not do that either.

push returns the length, not the list

None of us knew this:

```
> let todos = ["one", "two"]
> todos.push("three")
3
```

That `3` is the new length. Not the list, and not the item you added.

`push` changes the list it was given and hands back a number; `[...todos, x]` builds a new list and hands *that* back. Which is why one of them works with `setTodos` and the other does not.

What does React actually compare?

Asked in class, after “React sees no change if it is the same array”: same in what sense? The contents are different, so what is it looking at?

The answer is **identity** — whether it is literally the same array, rather than one that looks like it. JavaScript has no way to print an object’s identity, but `===` tests it, and that is enough to see the whole thing:

```
> let x = [1, 2]
> let y = [1, 2]
> x === y
false // identical contents, two different arrays

> y = x
> x === y
true // now they are one array with two names

> y = [...x]
> x === y
false // spreading built a new one
```

`===` on arrays and objects does not ask *are these the same to look at*. It asks *are these the same thing*. That last case is the one that matters: `[...x]` has the same contents and is a different array, which is exactly why it is the idiom you hand to a setter.

So React’s rule is not mysterious, it is cheap. Comparing contents would mean walking every to-do on every render; comparing identity is one check. The cost of that cheapness is the bargain you keep: give it a new array, or it will assume nothing happened.

You can try any of this in a terminal

You installed Node to run Vite, and it is also a JavaScript interpreter. Type `node` in a terminal and you get a prompt where you can test what a language feature actually does, without a browser, a component or a page refresh in the way:

```
$ node
> let todos = ["Buy milk"]
> todos.push("Call the landlord")
2
> todos
[ 'Buy milk', 'Call the landlord' ]
```

The browser's console does the same job and is one keystroke away while you are already looking at your app. Reach for either of them the moment you are *guessing* what a line does. Guessing is the slow way.

Something unclear in this section? [Open an issue](#).

Exam Questions

1. Explain the difference between props and state in React.
2. A component is handed `todos` as a prop and calls `todos.push("Buy milk")`. What happens on screen, what happens to the array, and what happens to the component that owns it?
3. What is *reactive* about programming in React?
4. You push a new item onto an array and the screen does not change. Why not, and what do you do instead?
5. Why should you avoid mutating state objects and arrays directly in React?

References

- *Adding Interactivity* > [State: A Component's Memory](#)

React Hooks

Source: [Lectures/Technical/React/Hooks.md](#) — something unclear? [Open an issue](#), or send a pull request.

You have just used one. `useState` is a hook, and it is worth stopping for a moment on what that word means, because every other one you meet this term follows the same rules.

A hook is how a component reaches things React is holding for it

Your component is a plain function. React calls it to get JSX, and then throws away every local variable inside it — that is why a normal `let count = 0` cannot survive a click.

But React itself remembers. It keeps a little store beside each component on screen, and a **hook** is how your function reaches into that store. `useState` asks for a slot that persists between calls; the setter tells React the slot changed, so the component should be called again.

That is the whole idea. Everything else is bookkeeping.

Something unclear in this section? [Open an issue](#).

They all start with `use`

Not decoration — it is how React's linter recognises them and checks the rules below. The three you will meet in this course:

hook	what it gets you	you reach for it
<code>useState</code>	a value that survives re-renders, and re-renders when it changes	today
<code>useEffect</code>	a way to reach <i>outside</i> React — storage, a backend, a timer	when the app has to stay in step with something React does not control
<code>useRef</code>	a value that survives re-renders but does not cause one	when you start extracting components

State and refs both survive a re-render; only state causes one. You want a ref for something the screen does not depend on — the DOM node you need to focus, a timer's id, a counter nobody displays.

Something unclear in this section? [Open an issue](#).

The rules: top level, always, unconditionally

- Call hooks at the **top level of a component**, not inside `if`, not inside a loop, not inside a nested function.
- They can also be called from another hook, which is how custom hooks work.

React will refuse to compile code that breaks this, and the reason is worth knowing, because it turns an arbitrary-looking rule into an obvious one.

React matches hooks to their stored values by call order, not by name. It does not know your variable is called `count`; it knows this component called a state hook first, and a second one after that. So:

```
function Broken({ showTitle }) {  
  if (showTitle) {  
    const [title, setTitle] = useState("My list"); // ← sometimes first, sometimes absent  
  }  
  const [items, setItems] = useState([]);          // ← so this is sometimes #1, sometimes  
    #2  
}
```

The first time `showTitle` is true, `items` is the second slot. When it turns false, the same line asks for the *first* slot — and gets handed the title. Everything after it shifts too. Keeping every hook at the top level, called every time, is what guarantees the order never changes.

Something unclear in this section? Open an issue.

Exam Questions

1. What are the rules for using React hooks?
2. Why does that rule exist — what would go wrong if you called a hook inside an `if`?
3. Both state and refs survive a re-render. What is the difference between them, and when would you reach for a ref?

References

- [Rules of Hooks](#) — the two rules, and what the linter checks
- [State: A Component's Memory](#)